czii-cryo-et-object-identification

11th place solution: batch normalization and 2D post-filtering

Rank	11
Team	MNISTery
Author	Jeroen Cottaar
Topic ID	561837
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561837
Retrieved with Kaggle CLI on 2026-10-21.

Code (training + inference, ready to submit): https://www.kaggle.com/code/jeroencottaar/czii-11th-place-solution

Our thanks to the competition organizers - this was a great introduction to 3D segmentation. And of course my own thanks to my teammate @davidlist !

Introduction

The goal of the competition was to identify various proteins in cryo-electron-tomography data (basically 3D maps of electron density), making this a 3D particle recognition problem. One key element was that the competition metric was heavily biased toward recall over precision - missing particles (false negatives) was far more expensive than predicting too many (false positives).

As most solutions, our core is a UNet segmentation model. We have added some elements that aren't commonly present in the other solutions:

- Batch normalization, stabilizing training and making inference consistent across window positions
- A post filter based on a 2D classification neural network, removing false positives for beta-galactosidase and thyroglobulin
- Predicting multiple ribosomes in close vicinity, allowing us to hit more ribosomes in confusing areas

What we really missed was to properly optimize our model on the public test dataset. Even though we used many submissions, those weren't really used in probing. We experimented quite a bit with UNet architectures and hyperparameters, but only in cross-validation, where it was almost impossible to draw any useful conclusions due to the small training set size.

In this writeup, we'll very briefly discuss the outline of our solution, and then focus on these 3 elements in turn.

Outline of solution

The core of our solution is a UNet:

- 32 models ensembled, with a mixture of ResNet architectures. No selection on CV or LB; we just train 32 times with different seeds.
- Models are ensembled by taking the mean of the heatmaps before softmax.
- All normalization is BatchNorm (details below).
- Training is always 2200 epochs with no early stopping, with learning rate reduced towards the end
- Augmentation: 90-degree rotations around Z axis, flipping in Y axis. Others didn't help.
- Loss: weighted combination of cross-entropy and Tversky loss (alpha=0.5, beta=8).
- No test time augmentation.

To go from the UNet segmentation map to particle locations:

- Apply thresholding and identify connected components.
- Take the centroids of each cluster as a prediction. If a cluster is very big, make multiple predictions (details below).
- Select which particles to include based on the connected cluster size. For beta-galactosidase and thyroglobulin, apply an additional filter based on a 2D neural network classifier (details below).

Batch normalization

For all normalization layers in the UNet, we use batch normalization. This means that rather than normalizing each instance, the normalization factors are based on running averages found during training (and fixed during inference). This significantly improves score and training stability. Why?

Imagine we're doing inference with either the red or the blue window below, and consider what happens at location X if we use anything other than batch normalization



If we're using the red window, the artifact on the bottom right has a large impact on our prediction at X, since the normalization is based on all data in the window. When we use the blue window, we suddenly get a very different prediction at X. While some type of long-range influence can be beneficial in a neural network, it should be clear that this one is entirely spurious. Note that this also means we get discontinuities at the transitions between inference windows (even when discarding edge pixels).

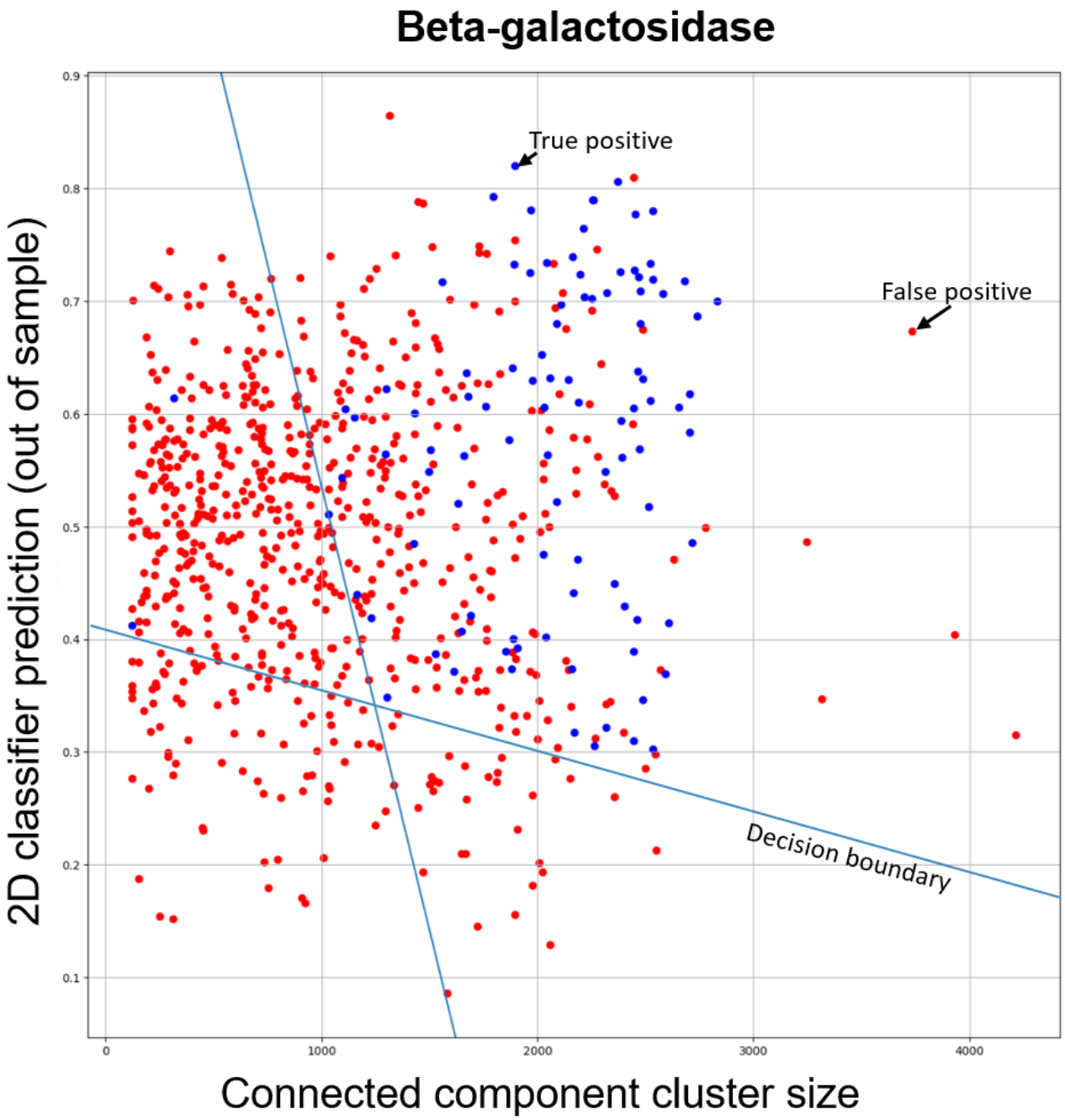
In MONAI specifically you can easily change from instance to batch normalization by adding "batch='NORM'" to the constructor, though you may have to mess with the momentum as well to get stable training (see our code for details).

2D classifier

As described in the competition paper, the hosts included classification based on 2D images. This inspired us to try the same.

We train a 2D neural network on the true and false positives of the UNet model (summing over the Z axis). The simplest architecture worked best: simply 3 fully connected layers (with dropout). For beta-galactosidase we add a single convolution layer at the start (without activation). Typical augmentations are applied, and some work is done to balance the true and false positives (see code for details).

As we can see below for beta-galactosidase, this allows us to filter out quite some false positives (compared to using cluster size alone as a feature). The decision boundary does need to be tuned on the leaderboard, which we unfortunately didn't do enough of.



Improved ribosomes

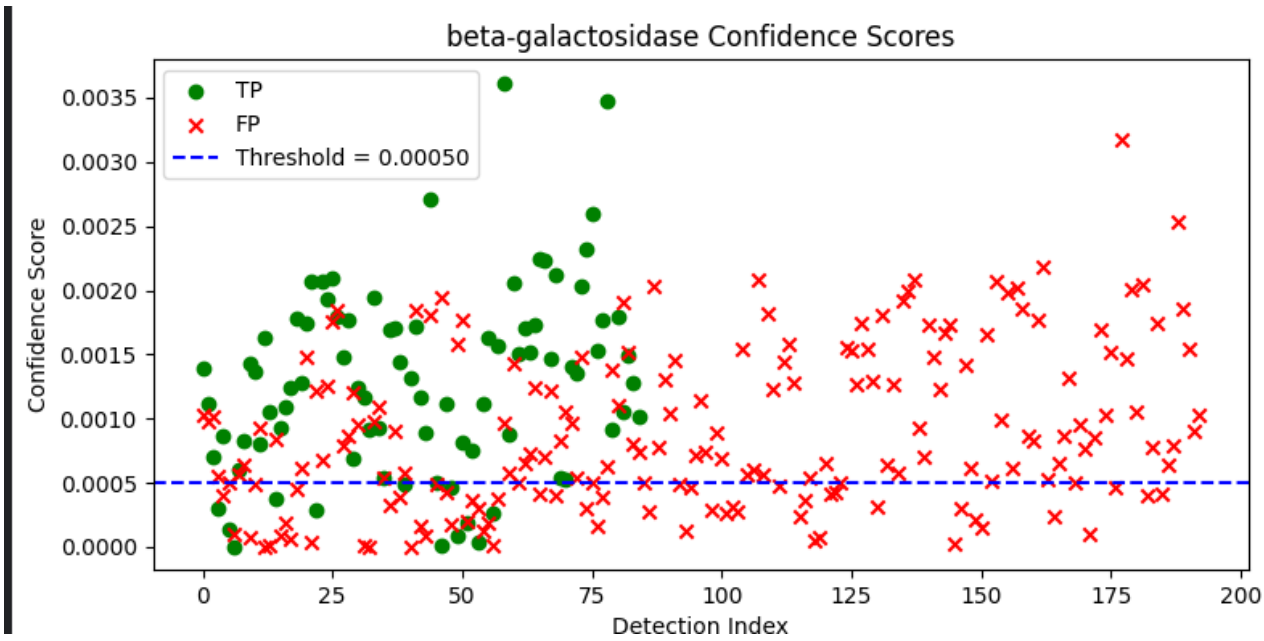
This is perhaps more of a 'Kaggle thing' (as in something you wouldn't typically add to a production machine learning solution)…

We noticed that we were struggling with clusters of ribosomes, and single ribosomes surrounded by other stuff. This is because clusters tend to melt together. We solve this by predicting multiple ribosomes for large, elongated clusters, using K-means to split up the cluster

Supplemental Q&A

> **IAmParadox · 2025-02-12T05:50:08.650000**

Nice work @jeroencottaar . I really enjoy reading your solution. Could you please share how much 2D NN classifiers for beta-galactosidase and thyroglobulin boosted your score? I was experimenting with something similar, we were checking the mean probability of the center pixels of the predicted blob something like 2x2x2 cube of beta-galacotosidase and try to find a confidence score for that



.

I ruled out classification option because I thought for the classifier to work it should have an accuracy of > 94%, which I thought was quite difficult for hard particles.

Jeroen Cottaar · 2025-02-12T15:17:12.803000

Thanks for your interest! It's hard to estimate the impact of the 2D classifier exactly, because turning it off requires reoptimizing other thresholds. But from what we saw throughout the competition, I think it's gaining 0.01 to 0.02.

Note that we don't try to select particles based on the 2D classifier score alone; we combine it with our existing selection mechanism (cluster size from the UNet).